Автоматическая генерация и бюджетный отбор признаков для рекомендательных систем

Смола Максим * **OpenAI Codex** **

* *Московский физико-технический институт, Москва, Российская Федерация*
** *ChatGPT University, San Francisco, California, USA*

Abstract: Предложен прототип автоматического feature engineering для рекомендательных систем. Генератор комбинирует сущности, статистики и временные окна, а bandit-inspired алгоритм последовательно отбирает признаки по приросту nDCG@10. Вычислительные затраты ограничиваются двухэтапной оценкой: дешёвым скринингом кандидатов и полной проверкой финалистов. На подвыборке MINDsmall выбран набор из пяти признаков против 24 у ручного baseline. При повторном обучении на пяти сидах средний nDCG@10 на dev составил 0,42986 против 0,42444 у baseline. Обсуждаются ограничения эксперимента и направления развития алгоритма.

1. ПОСТАНОВКА ЗАДАЧИ

Наша основная цель — исследовать возможность автоматизации построения и отбора признаков для рекомендательных систем. Традиционно для решения задачи ранжирования объектов используются модели градиентного бустинга, обученные на вручную сконструированных признаках, описывающих пользователей, объекты и историю их взаимодействий. Однако выбор информативных признаков, способов агрегации и временных окон требует значительных усилий и во многом зависит от экспертных решений.

В рамках этой работы предлагается алгоритм, вдохновлённый многорукими бандитами, а отчасти — и этой статьёй. Эксперименты были проведены на MINDsmall, потому что то ли задание было отредактировано, пока я над ним работал, то ли я просто пропустил пункт с предложенными датасетами.

> **Дисклеймер**
>
> Далее следуют описание алгоритма и результаты эксперимента. Читателей, чувствительных к нейрослопу, просим отойти от экранов.

2. ПРЕДЛОЖЕННЫЙ АЛГОРИТМ

2.1 Генерация кандидатных признаков

Один временной признак задаётся рецептом: группировка сущностей, статистика и временное окно. В реализации используются:

- пять группировок: user, item, user–item, user–category, user–subcategory;
- три статистики: число показов, число кликов и CTR;
- пять окон: 6 часов, 1 день, 3 дня, 7 дней и вся доступная история.

Таким образом, генератор создаёт $5 \times 3 \times 5 = 75$ временных рецептов. Дополнительно доступны десять признаков из исходных полей MIND: длина истории, наличие объекта в истории, число уникальных категорий и подкатегорий, число и доля исторических кликов по категории и подкатегории объекта, длины заголовка и аннотации. Всего получается 85 кандидатов.

Например, для объекта i CTR за окно w определяется как

$$\mathrm{CTR}_{i,w}(t) = \frac{\sum_{j:i_j=i,\,t-w\le t_j<t} y_j}{\sum_{j:i_j=i,\,t-w\le t_j<t} 1}. \tag{1}$$

При отсутствии показов значение равно нулю. Все события с текущей временной меткой исключаются, включая остальные строки той же выдачи. Окно all не имеет левой временной границы. История кликов, поставляемая MIND отдельным списком, не содержит индивидуальных временных меток: её признаки считаются по всему списку, а не по выбранным окнам.

До поиска удаляются константные признаки и точные дубликаты. Сравнение проводится только на поисковой части train. В нашем запуске исключено 17 кандидатов, осталось 68. Категория и подкатегория объекта всегда включены в модель; максимальный набор содержит 70 признаков.

Человек по-прежнему задаёт язык рецептов и допустимые окна. Автоматизируется построение комбинаций и выбор итогового подмножества, а не изобретение произвольных признаков за пределами этого языка.

2.2 Руки, награды и горизонт

В терминах многоруких бандитов **рука** — рецепт кандидатного признака. Её проверка означает обучение модели с добавлением этого признака к текущему набору S. **Награда** — парная разница качества:

$$r(a; S, f, s) = Q_{f,s}(S \cup \{a\}) - Q_{f,s}(S), \qquad (2)$$

где Q — nDCG@10, f — временной фолд, s — seed модели. Сравниваемые модели используют одинаковые фолды, seed и параметры обучения. После добавления признака набор S меняется, поэтому полезность остальных рук также может измениться. Награды нельзя считать стационарными и независимыми.

Горизонт здесь интерпретируется как бюджет проверок: максимальное число поисковых и контрольных обучений CatBoost. Финальный fit в бюджет не входит. Проверки с 50 и 300 деревьями учитываются как одно обучение каждая, поэтому дополнительно сохраняется общее число построенных деревьев.

Реализация — эвристика, вдохновлённая распределением бюджета между руками, а не классический MAB-алгоритм с UCB или Thompson Sampling. Доверительные границы наград и теоретические гарантии regret не выводятся. Поиск плоский по библиотеке рецептов; два уровня точности оценки не делают его иерархическим поиском группировок и окон.

2.3 Последовательный отбор

Перед поиском train сортируется по времени выдач. Ранние 80% выдач используются для поиска признаков, поздние 20% — для контроля остановки и выбора лучшего шага. Одинаковые временные метки не разделяются границей. На ранней части строятся три expanding-window фолда: примерно 0–40% → 40–60%, 0–60% → 60–80% и 0–80% → 80–100%. Эти проценты относятся к поисковой части, а не ко всему train.

Алгоритм начинает с категории и подкатегории и выполняет следующие действия:

1. **Скрининг.** На последнем поисковом фолде обучается текущий набор и его расширение каждой оставшейся рукой. Используются 50 деревьев и один seed. По дешёвому приросту выбираются три лучших кандидата.
2. **Полная проверка.** Каждый финалист оценивается с 300 деревьями на трёх фолдах и четырёх сидах: 42, 0, 1, 2. Награда усредняется по 12 оценкам.
3. **Добавление.** Выбирается кандидат с наибольшим средним приростом выше 0,001, если усреднённая по сидам награда положительна хотя бы на двух фолдах. При отсутствии такого кандидата поиск завершается.
4. **Контроль.** Только победивший набор проверяется на поздних 20% train с seed 42. Чекпоинт обновляется, если прирост относительно сохранённого лучшего набора превышает 0,001.
5. **Остановка.** После двух шагов без достаточного улучшения контроля возвращается лучший сохранённый набор. Поиск также ограничен бюджетом и максимумом 15 добавлений.

Перед новым раундом проверяется, достаточно ли бюджета на его полное выполнение. Бюджет — верхняя граница, а не обязательное число проверок. После выбора чекпоинта модель заново обучается на всём train.

Такой механизм сокращает число дорогих оценок: все оставшиеся руки получают дешёвую проверку, а многократное полное обучение получают лишь финалисты. При этом текущая реализация заранее вычисляет всю библиотеку признаков: экономия относится прежде всего к оценке кандидатов, а не к их материализации.

3. ЭКСПЕРИМЕНТ

3.1 Данные и протокол

Использован MINDsmall: каждая исходная запись содержит пользователя, время, историю кликов и список показанных новостей с бинарными метками. После разворачивания списка одна строка соответствует одному показу новости. Несколько строк с одинаковым impression_id образуют одну выдачу.

Взяты первые 3000 исходных выдач из train и первые 3000 из dev. Это подвыборки в порядке исходных файлов, а не случайная выборка и не первые 3000 выдач после сортировки по времени. Далее временные разбиения строятся по timestamp.

Table 1. Объём экспериментальной подвыборки.

Часть	**Выдач**	**Строк**	**Клики**
Train	3000	114 257	3,97%
Dev	3000	110 621	4,11%

Train охватывает 9–14 ноября 2019 года, dev — 15 ноября. Параметры поиска и чекпоинт выбираются внутри train; dev не передаётся в функцию отбора.

Используется **последовательный rolling-протокол**: исторические агрегаты для поздних строк validation или dev могут учитывать метки более ранних событий той же части. Это предполагает, что обратная связь уже поступила к моменту следующего события. Модель при этом не дообучается на dev. Такой протокол не эквивалентен статическому тесту с полностью скрытыми dev-label. Одинаковая схема применяется ко всем сравниваемым наборам.

Поскольку dev просматривался в процессе разработки, он является внешней проверочной выборкой, но не полностью нетронутым финальным тестом. Результаты следует рассматривать как проверку работоспособности идеи на ограниченном эксперименте.

3.2 Модель и сравниваемые наборы

Используется CatBoostClassifier: 300 деревьев, глубина 8, learning rate 0,05, функция потерь Logloss. Таким образом, обучение остаётся классификационным, а выбор признаков ориентирован на ранжирование. Время и идентификаторы выдач не подаются в модель как признаки.

Сравниваются три фиксированных набора:

- **Ручной baseline v1:** 24 признака — категории, характеристики истории и текстов, а также all-time показы, клики и CTR для user, item, user–category и user–subcategory.
- **AutoFE:** две фиксированные категории и признаки, выбранные алгоритмом.
- **Все кандидаты:** 70 признаков после фильтрации поисковой библиотеки.

Главная метрика — nDCG@10: DCG первых десяти позиций, нормированный на DCG идеального порядка. При бинарной релевантности

$$\mathrm{DCG@10} = \sum_{j=1}^{\min(10,m)} \frac{y_{\pi(j)}}{\log_2(j+1)}, \tag{3}$$

где π — порядок по предсказанию, m — размер выдачи. Метрика усредняется по выдачам с хотя бы одним кликом. Дополнительно рассчитаны MRR, nDCG@5, ROC-AUC, average precision (в коде обозначена PR-AUC) и LogLoss.

3.3 Результат отбора

При бюджете 1000 выполнено 533 оценочных обучения и построено 76 150 деревьев без финального fit. Весь запуск, включая генерацию и финальное обучение, занял около 20,6 минуты на использованной машине при thread_count = 4. Это измерение конкретного запуска, не оценка масштабируемости на больших данных.

Лучшим стал третий шаг. Итоговый набор содержит пять признаков:

1. категория новости;
2. подкатегория новости;
3. доля кликов по подкатегории новости в исходной истории пользователя;
4. CTR новости за предшествующие три дня;
5. доля кликов по категории новости в исходной истории пользователя.

Средний поисковый nDCG@10 выбранного набора равен 0,40242, контрольный — 0,33421. Эти величины используются при отборе и не являются независимой оценкой качества. На следующих двух шагах поисковая метрика продолжила расти, но контроль не улучшился на требуемые 0,001, поэтому алгоритм вернулся к набору третьего шага.

В основном запуске на dev nDCG@10 составил 0,43137 у AutoFE и 0,42639 у v1: абсолютный прирост около 0,00497. MRR вырос с 0,38093 до 0,38747, ROC-AUC — с 0,72248 до 0,72972, LogLoss снизился с 0,15904 до 0,15806.

3.4 Сравнение со случайными подмножествами

Чтобы проверить, достаточно ли просто взять мало признаков, из того же пула выбирались случайные подмножества разного размера. Две категориальные колонки всегда сохранялись, остальные выбирались без возвращения. Число признаков на оси включает эти две колонки. Для каждого размера выполнено пять повторов; полный набор из 70 признаков проверен один раз. Каждый набор обучался на всём train и оценивался на dev.

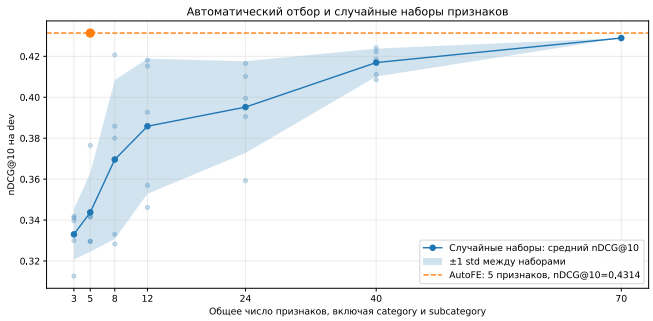


Fig. 1. nDCG@10 на dev в зависимости от числа признаков случайного подмножества. Ломаная отражает среднее по повторам; горизонтальная линия — качество выбранного AutoFE-набора из пяти признаков. График относится к основному запуску, а не к усреднению по пяти дополнительным сидам.

Средний nDCG@10 случайных наборов из пяти признаков составил 0,34375, из 24 — 0,39519, из 40 — 0,41695. Полный набор дал 0,42893. Выбранные пять признаков достигли 0,43137. Следовательно, в этом эксперименте важен состав набора, а не только его размер: случайный компактный набор не воспроизводит результат целенаправленного отбора.

3.5 Проверка на пяти сидах

Затем три набора фиксировались и повторно обучались с seed 11, 22, 33, 44, 55 при одинаковых остальных параметрах. Отбор признаков заново не запускался. В таблице приведены среднее и выборочное стандартное отклонение nDCG@10 на той же dev-выборке.

Table 2. Повторное обучение фиксированных наборов на пяти сидах.

Набор	Признаки	nDCG@10
AutoFE	5	0,42986 ± 0,00146
Ручной v1	24	0,42444 ± 0,00348
Все кандидаты	70	0,42576 ± 0,00212

AutoFE превосходит ручной набор на всех пяти сидах, полный — на четырёх из пяти. Средние парные разницы составили соответственно +0,00542 и +0,00410. Таким образом, выигрыш не ограничен единственным seed обучения. Однако эти повторы измеряют только случайность обучения модели: они не оценивают устойчивость самого поиска, смену данных или статистическую значимость на новых временных периодах.

4. КАК УЛУЧШИТЬ ИДЕЮ АЛГОРИТМА

Адаптивное распределение бюджета. Сейчас каждая оставшаяся рука заново проходит скрининг на каждом шаге, а число финалистов фиксировано. Можно вести оценки неопределённости парного прироста и добавлять фолды, сиды или деревья прежде всего кандидатам, между которыми пока нет уверенного различия. После изменения набора признаков старые оценки потребуют пересмотра: полезность руки зависит от контекста. Это приблизит метод к адаптивному

best-arm identification, но потребует отдельной проверки корректности доверительных границ.

Многоуровневая точность без жёсткого отсечения. Модель с 50 деревьями может недооценить признак, полезный при 300 деревьях. Вместо безусловного допуска только трёх лучших кандидатов можно использовать несколько ступеней бюджета и сохранять небольшую долю исследовательских проверок вне текущего топа. Это уменьшит риск раннего отбрасывания перспективного рецепта.

Иерархическая генерация и уточнение окон. Вместо полной материализации сетки можно сначала исследовать семейства группировок и статистик, а затем уточнять окна только у перспективных семейств. Например, после успеха item–CTR за три дня проверить соседние горизонты. Для такого поиска нужно сохранять возможность исследования остальных ветвей, иначе дешёвая ранняя оценка может ошибочно закрыть полезное семейство. Это уже будет развитие генерации, а не только отбора из фиксированного пула.

Совместная полезность признаков. Жадное добавление по одному признаку не обнаруживает пары, которые бесполезны по отдельности, но полезны вместе. Ограниченный поиск пар, beam search или операции удаления и замены позволят выходить из локально удачных, но неоптимальных наборов. Особенно важно разрешить удаление признака после изменения контекста.

Награда с учётом стоимости. Число fit не отражает разницу между 50 и 300 деревьями, а также стоимость расчёта разных агрегатов. Бюджет можно задавать в деревьях или времени, а качество оптимизировать совместно с затратами на материализацию и применение признака. Для практического применения полезен набор решений на границе «качество — стоимость», а не только единственный максимум nDCG.

Устойчивость к изменению временного режима. Один фиксированный поздний контрольный участок может определять результат слишком сильно. Для отбора можно оценивать не только средний прирост, но и его разброс по нескольким временным периодам, штрафуя нестабильные признаки. При этом участки, используемые для адаптивного выбора, должны оставаться отделёнными от финальной внешней оценки.